

Impact of Quantization and Structured Pruning on Battery SOH Prediction Models across Architectures

Jonas Schulze^a, Nils Strassenburg^a, Aleinikau Pavel^a, Florian Rzepka^{a,*}

^aTechnical University of Berlin, Chair of Electrical Energy Storage Technology, Einsteinufer 11, Berlin, 10587, Berlin, Germany

Abstract

- Core objective: investigate how pruning, quantization, and knowledge distillation affect SOH prediction quality and embedded deployment cost across LSTM, GRU, CNN, and TCN architectures.
- Study principle: architectures are trained under a harmonized protocol first, then optimized under controlled compression settings to enable fair cross-architecture comparison.
- Evaluation focus: MAE/RMSE on SOH prediction and embedded KPIs (flash, RAM, latency, energy).
- Expected outcome: architecture-specific design guidance for resource-constrained BMS implementations.

Keywords: Battery SOH prediction, Structured pruning, Quantization, Knowledge distillation, Embedded AI, LSTM, GRU, CNN, TCN, BMS

1. Introduction

With the increasing demand for clean energy, driven by environmental concerns, energy storage systems play a crucial role in achieving this goal [1, 2]. Given the high power and energy density required for these tasks, Lithium-Ion batteries appear to be the perfect match [3, 4]. Furthermore, Lithium-Ion batteries have a relatively high cycle life and a low self-discharge capability [5]. To ensure the safe and efficient operation of these systems, a battery management system (BMS) is used to measure and monitor voltage, current, and temperature, and to estimate State of Health (SOH) [6]. The SOH is defined as:

$$\text{SOH} = \frac{Q_{\text{current}}}{Q_{\text{initial}}} \quad (1)$$

Where Q_{current} and Q_{initial} are the current actual usable capacity and the initial capacity of the battery. The SOH is used as a metric for quantifying battery degradation [7], where 100% represents a new battery. As degradation progresses, the percentage decreases until a defined end-of-life [8]. Accurate SOH estimation remains a challenging task that has received significant attention in the research community [9, 10], especially under the strict compute and memory constraints of embedded BMS hardware [11]. There are different SOH estimation approaches that vary in complexity. Physics-based methods require computationally intensive partial differential equations to simulate the dynamic behaviour of a battery [12, 13]. Equivalent circuit models (ECM) are simpler and lighter models but rely on extensive experimental testing and prior knowledge [14, 15]. In recent years, data-driven methods have received increasing attention in SOH Estimation research [16]. These methods do not require knowledge of the battery's internal processes; instead, they aim to learn the ageing mechanism from historical data [9]. Particularly, the field of deep learning approaches involves different types of architectures, such as Long Short-Term Memory (LSTM), gated recurrent unit (GRU), convolutional neural networks (CNN) and temporal convolutional networks (TCN) [16, 17], which have been

*Corresponding author

Email address: florian.rzepka@tu-berlin.de (Florian Rzepka)

used for SOC and SOH estimation. However, deploying parameter-heavy deep learning architectures on resource-constrained microcontrollers requires optimisation techniques such as pruning and quantisation [18]. Tiny Machine Learning (TinyML) enables edge execution [19], but it is unclear how different deep learning architectures respond to compression techniques in regard to accuracy and inference time. This study investigates whether compression techniques are architecture-dependent. To develop and test the SOH estimation algorithms on potent hardware, we use the STM32H755ZI-class microcontroller. The practical challenge is not only prediction accuracy but also deployability under strict flash, RAM, latency, and energy limits. With this high-performance microcontroller, new state estimation algorithms can be tested and validated, enabling further investigation into TinyML state estimation. To develop and test these four models, the Lithium Iron Phosphate (LFP) DoE Cycle Ageing Dataset [20] will be used. In this dataset, cells have been long-term cycled under laboratory conditions, which include constant temperatures, regular check-ups and hybrid pulse power characterisation (HPPC) tests.

This paper is structured as follows: Section 2 reviews related work on the four architectures. Section 3 describes the experimental setup, divided into six parts: the comparison metric; the dataset and embedded target used; the model architecture portfolio; the training protocol; the model optimisation methods; and the embedded benchmark protocol.

- This study investigates whether compression effects are architecture-dependent and whether different model families remain equally usable after optimization.
- The paper therefore compares recurrent and convolutional architectures under one consistent pipeline from training to embedded benchmarking.
- Figure 1 (planned, position in this section): high-level study concept showing the flow from baseline models to compression methods and final embedded evaluation.

2. Related Work and Research Gap

Deep Learning approaches for estimating SOH can be divided into recurrent and convolutional networks. For recurrent models, LSTMs are often used because they can capture long-term temporal ageing patterns. For example, Nguyen Van et al. [21] developed a two-stage LSTM for SOH and internal resistance estimation, outperforming a reference Feedforward Neural Network (FNN) in accuracy. To improve accuracy and robustness, Xu et al. [22] incorporated data characteristics and spatio-temporal attention into an LSTM-based SOH-estimation method. Ungurean et al. [23] reduced the parameter footprint of a GRU using the adaptive moment estimation algorithm. Compared to an LSTM, the GRU had significantly fewer parameters and only a slightly higher estimation error, making it a suitable candidate for an embedded implementation on a BMS. Similarly, Vaishnani et al. [24] observed that a GRU achieved better performance (RMSE = 0.017) over LSTM, RNN, and hybrid LSTM-GRU models.

In contrast, convolutional architectures like the TCN enable parallel processing. Yang et al. [17] proposed a TCN with an attention mechanism, which achieved an average RMSE of 2.10%, outperforming a CNN, support vector regression, and a basic TCN. Furthermore, Bi et al. [25] showed that the TCN outperformed the LSTM by 3% in accuracy under accelerated ageing conditions. Additionally, Alberti et al. [26] evaluated an LSTM, a GRU, a one-dimensional CNN, and a hybrid CNN-LSTM; in that study, the one-dimensional CNN yielded the lowest estimation error and prediction variance.

Kim et al. [27] also propose an approach to developing a lightweight, edge-compatible deep learning framework for SOH estimation. They reduced the model size by 99% using knowledge distillation, structured pruning, and dynamic quantisation and outperformed a MobileNetV1-based lightweight baseline in both accuracy and compactness.

Although existing studies compare neural network architectures for SOH estimation, these comparisons are conducted on high-performance computing platforms and do not examine compression techniques such as pruning and quantisation. Consequently, it remains unexplored whether the advantages of deep learning approaches can withstand compression and operate in resource-constrained environments.

- Existing SOH studies report promising sequence-model performance, but many evaluations remain model-specific and not directly comparable across architectures.
- Compression literature often focuses on algorithmic gains in isolation and rarely combines battery-domain performance with deployment-level KPIs.

- The open gap is a controlled cross-architecture analysis that links compression decisions directly to BMS-relevant engineering costs.
- Figure 2 (planned, position in this section): literature positioning map contrasting accuracy-centric studies, compression-centric studies, and the proposed integrated approach.

3. Methodology

3.1. Comparison Objective and Benchmark Design

The comparison is designed to determine whether the effects of model optimization transfer consistently across neural architectures or depend on the mechanism used to represent temporal battery behavior. For this purpose, strong uncompressed baseline models are first established for the LSTM, GRU, CNN, and TCN families. Structured pruning and quantization are then applied under controlled conditions. The benchmark therefore examines not only whether these methods reduce model size and embedded execution cost, but also whether the resulting loss of SOH prediction accuracy follows a comparable pattern across architectures. This makes it possible to identify cases in which an expected compression benefit generalizes across model families and cases in which a particular architecture is more sensitive to parameter removal or reduced numerical precision.

The comparison is organized around the change from each architecture-specific FP32 baseline rather than around baseline accuracy alone. Prediction quality is evaluated using MAE and RMSE, while deployment cost is characterized through model footprint and the embedded metrics flash, RAM, latency, and energy. Model sizes may differ where this is required to establish competitive baseline quality. However, the data partition, target definition, preprocessing, error metrics, optimization settings, and benchmark protocol are held fixed wherever applicable. The resulting accuracy–resource trade-offs provide the basis for selecting optimization strategies according to both architecture and deployment constraints.

3.2. Data, Target, and Split Strategy

The dataset is based on experiments conducted with JGCFR18650-1800mAh-3.2V LFP cells [28] in a university laboratory and is publicly available as the LFP DoE Cycle Aging [20]. The cells were cycled inside a climate chamber at 25° under the following charging C_{chg} , discharging C_{dis} and Depth of Discharge (DoD) conditions:

$$C_{\text{chg}} \in [0.5C, 0.9C] C_{\text{dis}} \in [1C, 2.5C] \text{DoD} \in [45\%, 65\%] \quad (2)$$

This experiment used 15 test cells across eight operating conditions, with two cells per condition to ensure reproducibility. To monitor battery degradation, check-ups consisting of capacity tests, open-circuit voltage sweeps, and HPPC tests were performed after each aging loop. Furthermore, a load profile for a stationery storage unit was included to emulate realistic power flow [29]. The Data contains voltage, current, terminal temperature, auxiliary channels and is stored in Parquet tables.

- Prediction target: battery SOH over aging trajectories.
- Feature basis from current code: Voltage, Current, Temperature, EFC, and Q_c .
- Feature preprocessing: hourly aggregation into mean, std, min, and max channels.
- Common partitioning: train cells C01/C03/C05/C09/C13/C17/C25/C27/C29; validation cells C07/C15/C21; test cells C11/C19/C23.
- Figure 3 (planned, position in this subsection): SOH trajectories of all cells with color-coded train/validation/test assignment.

This split figure documents partition coverage and helps verify that all sets contain representative SOH evolution ranges.

Table 1: Architecture comparison for the SOH optimization study. Sequence lengths are given in hourly steps, and FP32 memory assumes four bytes per trainable parameter.

Family	Input steps	Architecture	Parameters	FP32 weights
LSTM	192	Feature projection, 2 recurrent layers ($H = 192$), 3 residual MLP blocks	895,937	3.418 MiB
GRU	168	Feature projection, 3 recurrent layers ($H = 160$), 4 residual MLP blocks	844,321	3.221 MiB
CNN	96	3 residual causal blocks ($C = 128, k = 5, d = 1/2/4$)	503,297	1.920 MiB
TCN	120	5 residual temporal blocks ($C = 96, k = 5, d = 1/2/4/8/16$)	439,841	1.678 MiB

3.3. Architecture Portfolio and Baseline Selection

The portfolio comprises two recurrent architectures, an LSTM and a GRU, and two convolutional architectures, a CNN and a TCN. This selection enables compression to be compared across different mechanisms for temporal representation instead of within a single model family. All four models receive the same 20-channel input representation obtained from the hourly mean, standard deviation, minimum, and maximum of voltage, current, temperature, EFC, and cumulative charge. One SOH target is assigned to each hourly interval. Each model is causal and produces one SOH estimate at every time step.

The recurrent models use a learned feature projection followed by a unidirectional recurrent stack, layer normalization, residual MLP blocks, and an MLP output head. The selected LSTM contains two recurrent layers with a hidden size of 192 and three residual blocks. The GRU uses three recurrent layers with a hidden size of 160 and four residual blocks. The convolutional models process the same sequences using residual causal convolutions. The CNN contains three blocks with 128 channels, a kernel size of five, and dilation factors of 1, 2, and 4. The TCN extends this dilation pattern to 1, 2, 4, 8, and 16 across five blocks with 96 channels, thereby providing a wider temporal context without recurrent state propagation. Table 1 summarizes the selected uncompressed FP32 baselines.

Baseline selection followed two stages. First, a family-specific hyperparameter search determined a candidate configuration using validation RMSE. Second, a local capacity sweep evaluated coordinated reductions and increases in the internal widths while retaining the remaining architecture and training settings. The investigated hidden or channel widths were 64–256 for the LSTM, 48–256 for the GRU, 64–224 for the CNN, and 48–144 for the TCN. The final reference configuration was selected from this sweep by validation performance, while the test cells were reserved for the subsequent comparison. This procedure checks the neighborhood of the selected capacity and avoids basing the compression study on an arbitrary model size.

3.4. Training and Evaluation Protocol

All architectures were trained with the same cell-level partitioning and random seed of 42. A robust scaler was fitted exclusively to the training cells and subsequently applied without refitting to the validation and test cells. The hourly feature vectors were arranged as overlapping causal sequences with a stride of one. Their architecture-specific lengths are listed in Table 1. Each model was trained in a sequence-to-sequence setting and therefore returned one SOH estimate for every hourly step. The first 8 steps were excluded from the loss for the recurrent models, compared with 12 steps for the CNN and 16 steps for the TCN, to reduce the influence of state initialization and incomplete convolutional context.

The training objective combined the mean squared prediction error with a temporal smoothness penalty,

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 + \lambda_s \mathcal{R}(\Delta \hat{y}), \quad \Delta \hat{y}_t = \hat{y}_t - \hat{y}_{t-1}, \quad (3)$$

where $\mathcal{R}$ denotes the mean absolute step change for the LSTM, CNN, and TCN and the mean squared step change for the GRU. The smoothness term discourages implausible hour-to-hour fluctuations without imposing a fixed degradation trajectory. AdamW was used for parameter optimization, followed by a cosine-annealing schedule with warm

Table 2: Training settings of the selected FP32 baseline models. The epoch entry gives the maximum number of training epochs, and λ_s denotes the weight of the temporal smoothness penalty.

Family	Batch size	Epochs	Learning rate	Weight decay	λ_s	Penalty
LSTM	128	60	9.28×10^{-5}	3.49×10^{-4}	0.0830	L1
GRU	128	60	2.81×10^{-4}	2.60×10^{-5}	0.0051	L2
CNN	48	50	3.23×10^{-4}	3.76×10^{-5}	0.0843	L1
TCN	64	60	1.97×10^{-4}	5.88×10^{-5}	0.0480	L1

restarts. Gradient clipping and early stopping limited unstable updates and unnecessary training. Table 2 reports the architecture-specific settings retained after the hyperparameter search.

Validation was performed every three epochs. The checkpoint with the lowest validation RMSE was retained, with an early-stopping patience of 12 epochs for the LSTM, GRU, and TCN and 10 epochs for the CNN. The held-out test cells were used only after model and capacity selection. Prediction quality was quantified by

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|, \quad \text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}. \quad (4)$$

The metrics were first calculated for each test cell and then combined using the number of samples per cell as weights. All errors are reported on the dimensionless 0–1 SOH scale. Accordingly, an MAE or RMSE of 0.01 corresponds to one SOH percentage point on the 0–100% scale.

3.5. Model Optimization Methods

Pruning. Pruning, in the scope of our work, is the process of removing weights from a model. Typically, the model components that contribute the least to a model’s performance are candidates for removal. Pruning can be classified as either *unstructured* or *structured*. In unstructured pruning, we pinpoint the exact, individual weights we seek to remove. As the name suggests, the resulting model loses its structure; thus, to keep the matrices in an integral form, the selected weights are simply set to zero. We achieve high sparsity with minimal loss in accuracy, but the irregular matrices do not run faster on standard hardware that does not support sparse execution. In structured pruning, entire weight clusters, such as matrix rows/columns, filters, and channels, are removed. This physically shrinks the model architecture, making it faster and more efficient on typical hardware but usually comes with a sharper drop in accuracy than unstructured pruning.

Quantization. Quantization refers to reducing the precision of model parameters, usually the weights. Typical models are stored in FP32 (32-bit floating point), meaning each parameter uses 32 bits to represent it. When quantized, these parameters would then only be saved with e.g., 16 (FP16), 8 (INT8) or 4 (INT4) bits. This directly results in smaller file sizes. If the underlying hardware supports the quantized representation, these models can be executed significantly faster at minimal accuracy loss. Quantization can be paired with pruning to leverage both techniques. In this case, the model is usually pruned first, and the resulting, sparse model is quantized afterward.

- Figure 6 (planned, position in this subsection): optimization pipeline diagram showing baseline model, individual optimization branches, and combined branch.

3.6. Embedded Benchmark Protocol

- All variants are benchmarked with the same target platform and firmware toolchain.
- Inference is performed with batch size 1 and fixed warm-up, because deployment is stateful and sample-by-sample in online BMS operation.
- Embedded KPIs: MAE, flash, RAM, latency (mean and std), and energy.
- Quantization path in current code: dynamic INT8 for LSTM/GRU, static FX INT8 for CNN, and stabilized head-focused static FX for TCN.

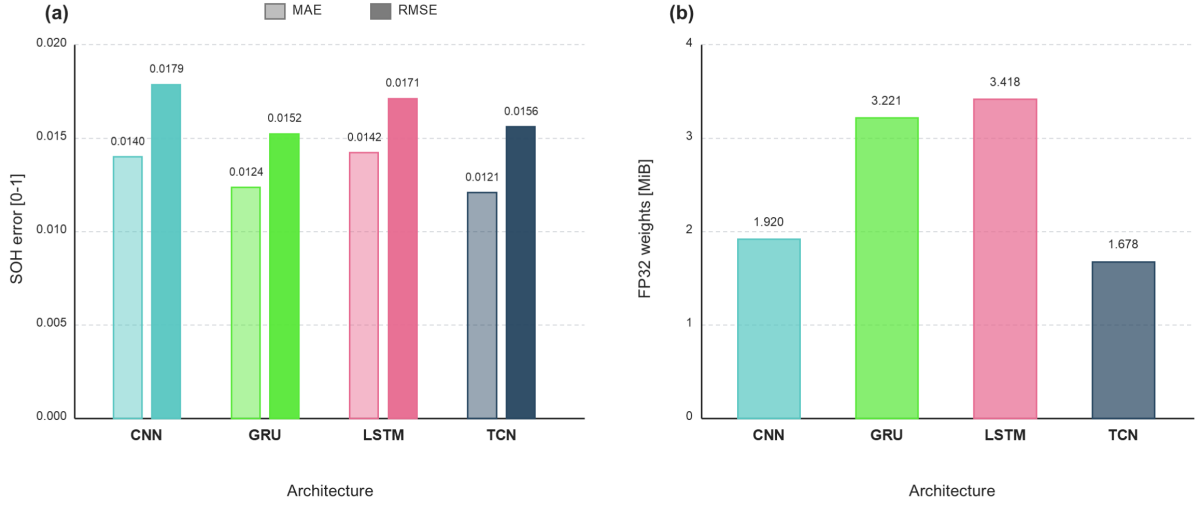


Figure 1: Comparison of the uncompressed FP32 baseline models. (a) Sample-weighted MAE and RMSE on the held-out test set using the dimensionless 0–1 SOH scale. A value of 0.01 corresponds to one SOH percentage point. (b) Parameter-only FP32 memory calculated with four bytes per trainable parameter.

- Figure 7 (planned, position in this subsection): benchmark setup including host pipeline, firmware execution, and metric logging flow.

4. Results

4.1. Baseline Model Comparison

The uncompressed FP32 models were first compared independently of any optimization. Figure 1(a) reports sample-weighted errors on the held-out test set using the 0–1 SOH scale defined in Section 3.4. The four architectures lie within a narrow accuracy range, with MAE values between 0.0121 and 0.0142 and RMSE values between 0.0152 and 0.0179. The TCN achieves the lowest MAE, whereas the GRU yields the lowest RMSE, showing that the baseline ranking depends on the selected error metric.

The parameter footprints differ more clearly than the prediction errors. As shown in Figure 1(b), the TCN and CNN require 1.678 and 1.920 MiB of FP32 weights, respectively, compared with 3.221 MiB for the GRU and 3.418 MiB for the LSTM. The largest baseline is therefore approximately twice the size of the smallest one despite their similar prediction accuracy. These differences provide distinct starting points for evaluating the architecture-dependent effects of compression in the following sections.

Figure 2 complements the aggregated errors with a trajectory-level comparison for one of the held-out test cells. All selected baseline models reproduce the long-term SOH decline and the principal local variations. Their deviations from the reference nevertheless differ over time, which explains why the aggregate ranking cannot be inferred from a single operating interval. This single-cell view is therefore illustrative, while Figure 1(a) remains the quantitative comparison across the complete test set.

The local capacity sweep provides an additional sensitivity check around the initial configuration obtained for each architecture. Figure 3 compares coordinated reductions and increases in the internal widths on the same held-out test-cell trajectory. The relationship between parameter count and prediction error is non-monotonic across all four families. Increasing capacity therefore does not guarantee a lower error, while some compact variants remain competitive with substantially larger networks. This observation supports architecture-specific capacity searches rather than selecting the largest feasible model by default.

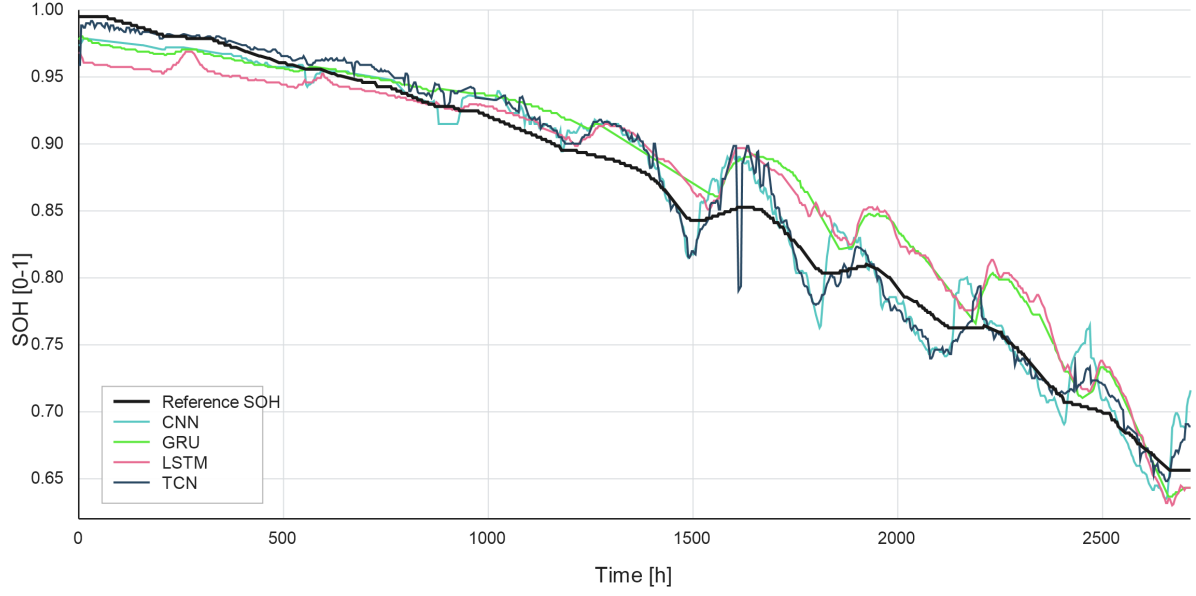


Figure 2: SOH trajectories of the selected uncompressed FP32 baseline models for one of the held-out test cells over 2716 hourly samples. The black line denotes the reference SOH, and the colored lines denote the predictions of the four architecture families.

Only MAE is shown in Figure 3 to keep the four-family comparison legible. The single-cell analysis is used to characterize local size sensitivity, whereas the final reference configurations were selected using validation RMSE as described in Section 3.3 and are compared across all held-out test cells in Figure 1.

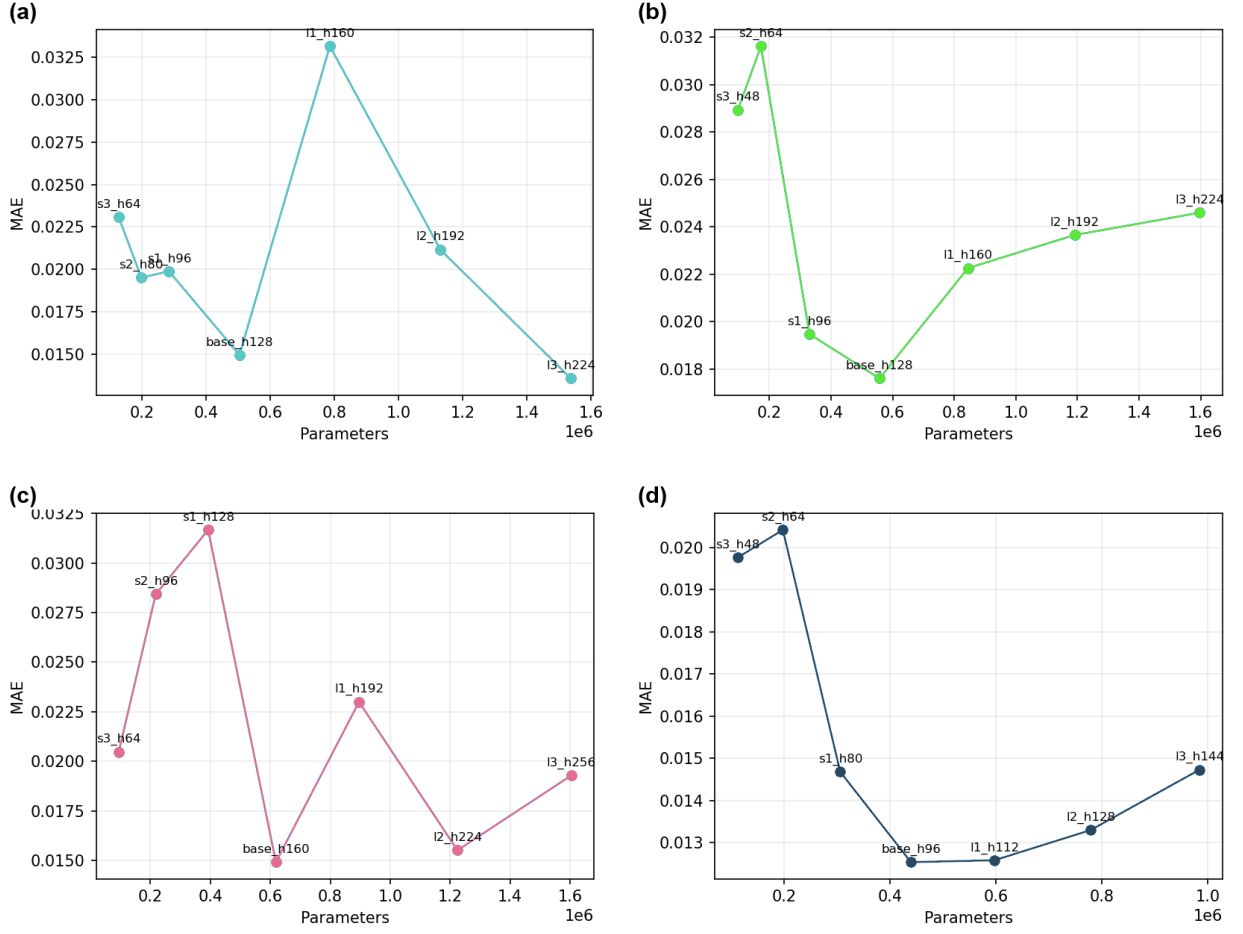


Figure 3: Local capacity sensitivity on one held-out test-cell trajectory for (a) CNN, (b) GRU, (c) LSTM, and (d) TCN. The label *base* denotes the initial configuration around which coordinated smaller (s1–s3) and larger (l1–l3) variants were generated. MAE is shown on the normalized 0–1 SOH scale, where 0.01 corresponds to one SOH percentage point.

4.2. Pruning Results

- Report MAE/RMSE trends over pruning levels per architecture.
- Report flash/RAM/latency/energy changes over pruning levels.
- Identify pruning thresholds where performance leaves the acceptable operating range.
- Figure 9 (planned, position in this subsection): pruning sensitivity curves and per-architecture Pareto view.

4.3. Quantization Results

- Compare FP32 and lower precision variants per architecture.
- Quantify trade-offs between prediction quality and embedded resource gains.
- Distinguish robust quantization cases from degradation-prone cases.
- Figure 10 (planned, position in this subsection): precision-vs-MAE and precision-vs-resource multi-panel plot.

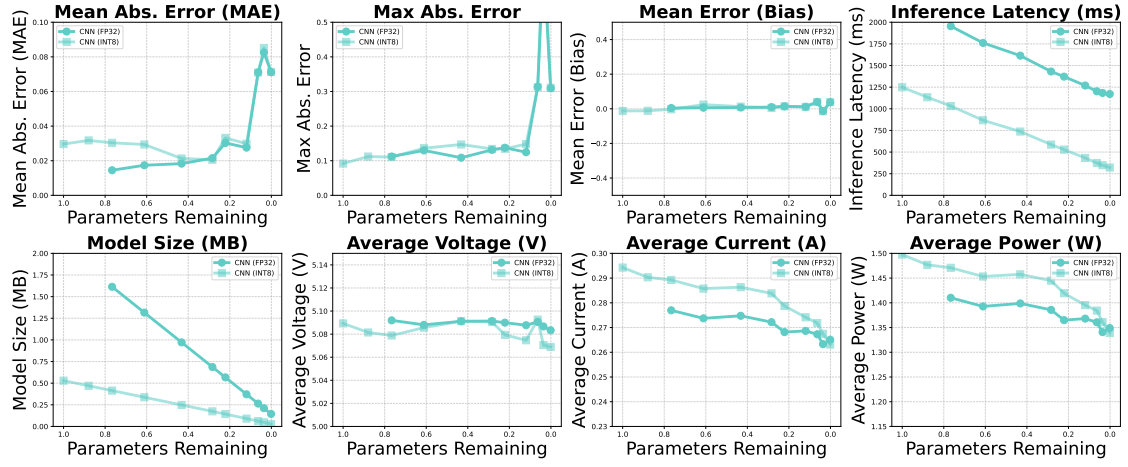


Figure 4: CNN metrics.

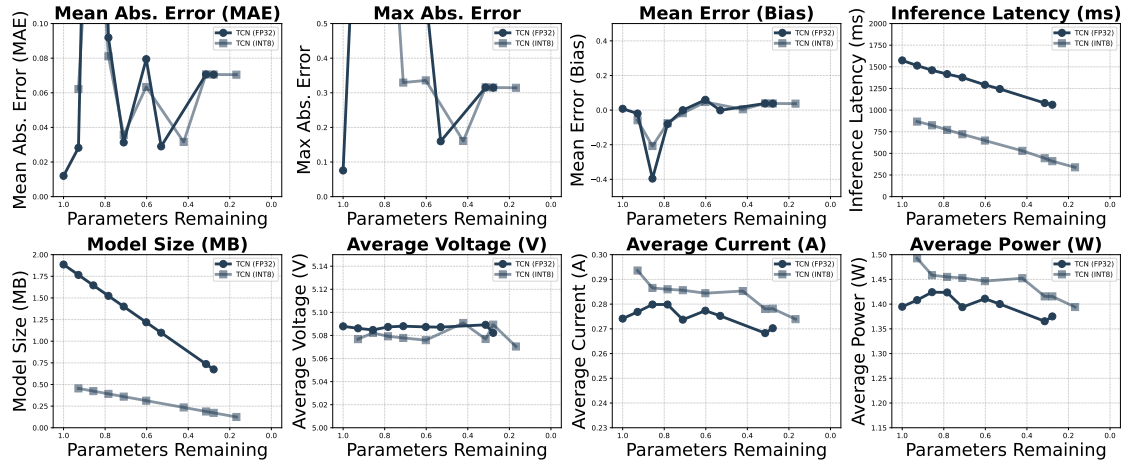


Figure 5: TCN metrics.

4.4. Distillation and Combined Optimization Results

- Evaluate whether distillation recovers accuracy for compressed student models.
- Compare single-method and combined-method variants within one decision space.
- Analyze additive versus interaction effects between optimization methods.
- Figure 11 (planned, position in this subsection): combined optimization scatter with non-dominated configurations.

4.5. Cross-Architecture Decision View

- Summarize all accepted variants in architecture-wise and global Pareto fronts.
- Highlight architecture-dependent operating regions for low-memory, low-latency, and low-energy constraints.
- Figure 12 (planned, position in this subsection): final decision map for architecture and optimization selection.



Figure 6: GRU metrics.



Figure 7: LSTM metrics.

5. Discussion

5.1. Baseline Architecture Comparison (Flo, EET)

The baseline comparison shows that architecture choice affects resource demand more clearly than SOH prediction accuracy. Although all four models operate within a narrow MAE and RMSE range, their FP32 parameter footprints differ by approximately a factor of two. The TCN combines the lowest MAE with the smallest baseline footprint, whereas the GRU achieves the lowest RMSE at a larger model size. Consequently, no architecture is uniformly superior across error metrics and storage requirements. The appropriate baseline depends on whether average absolute deviation, stronger penalization of larger errors, or memory demand is prioritized by the application.

The local capacity sweeps further show that increasing the number of parameters does not produce a monotonic accuracy improvement. Compact configurations can remain competitive, while larger variants may generalize less effectively despite their additional capacity. This finding is important for the subsequent optimization comparison because it separates compression effects from arbitrary over-sizing of the original network. Each architecture must first be evaluated near a suitable capacity before the effect of pruning or quantization can be interpreted fairly. The trajectory comparison supports this conclusion at the single-cell level: all selected models reproduce the long-term degradation trend, but their local deviations occur at different points in time. Aggregate metrics across all test cells therefore remain necessary for the final ranking.

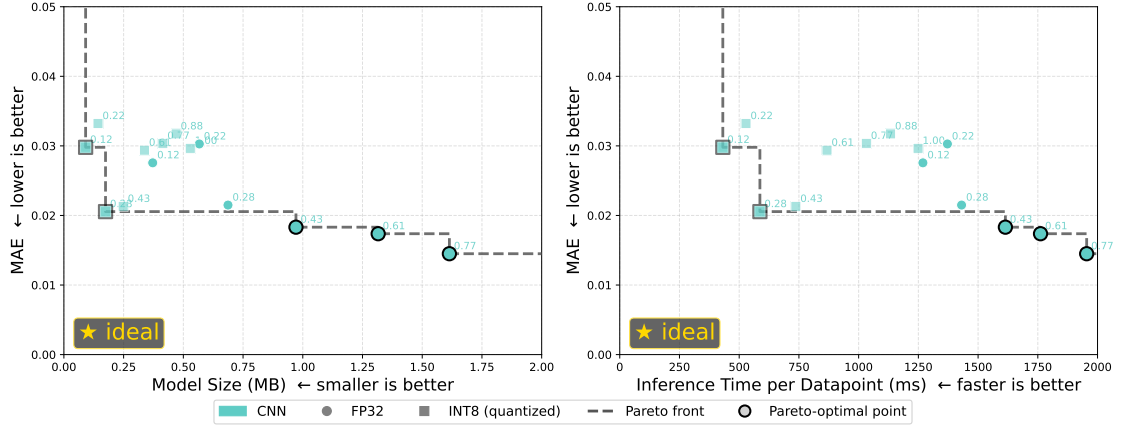


Figure 8: CNN pareto.

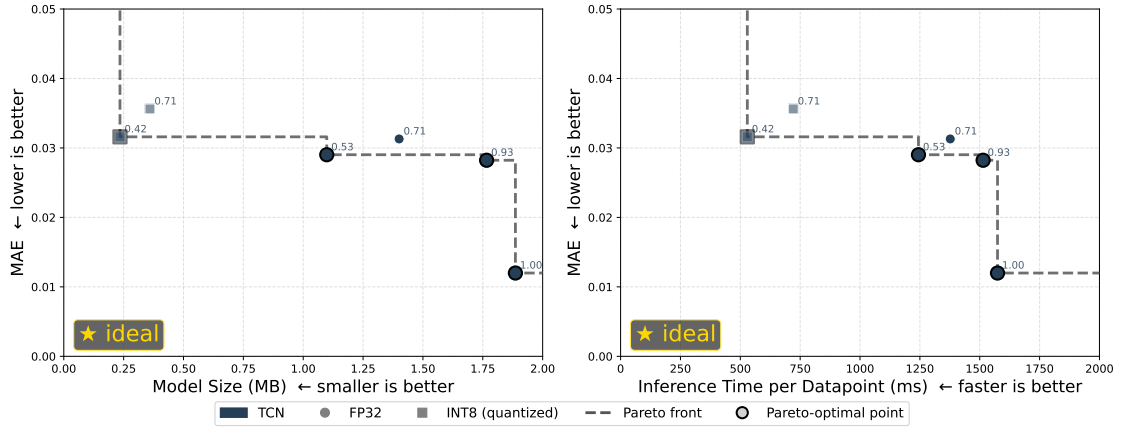


Figure 9: TCN pareto.

5.2. Cross-Architecture Effects of Optimization (HPI)

- Interpret why architectures react differently to identical optimization settings.
- Discuss where higher model complexity is justified by robustness gains and where lightweight structures dominate.
- Clarify methodological limits (data regime, target platform scope, and configuration breadth).
- Figure 13 (planned, position in this section): qualitative decision matrix linking constraints to recommended architecture-method combinations.

6. Conclusion and Outlook

6.1. Summary of Optimization Results (HPI)

- The study provides a controlled architecture-level comparison of pruning, quantization, and distillation for SOH prediction.
- The central contribution is a reproducible accuracy-cost analysis that directly supports embedded design decisions.
- Future work extends the study to additional chemistries, broader operating regimes, and tighter hardware-in-the-loop integration.

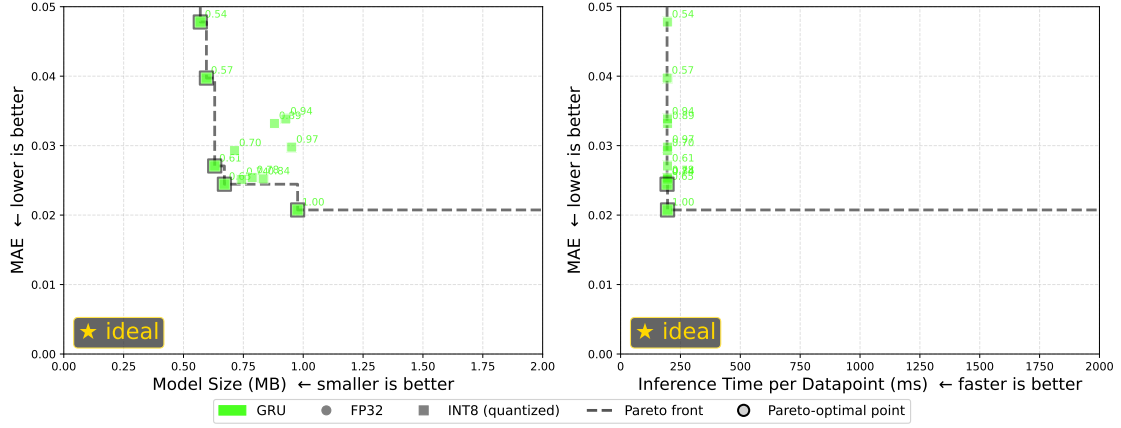


Figure 10: GRU pareto.

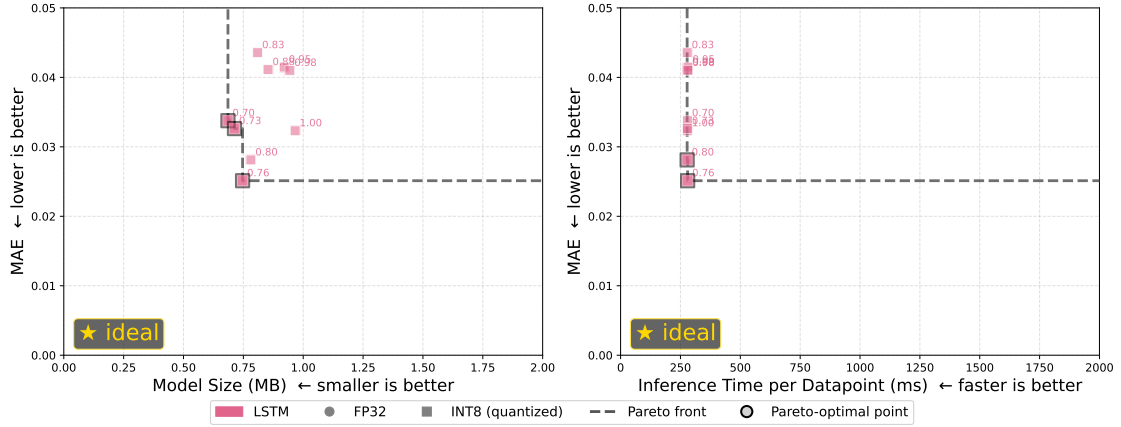


Figure 11: LSTM pareto.

6.2. Implications for Battery Management Systems (Flo, EET and Pavel, EET)

For a BMS, optimization is useful only when the reduction in resource demand preserves an SOH estimate that remains sufficiently accurate for monitoring, maintenance planning, and operating-limit adaptation. The combined structured-pruning and INT8-quantization variants reduce parameter storage by approximately 80.0–81.5% for the CNN, GRU, and LSTM, while the absolute change in normalized MAE remains below 0.0008. These results indicate that substantial storage reductions are possible without a corresponding increase of the same magnitude in estimation error. The small accuracy improvements observed for some variants should not be interpreted as a general benefit of compression, but rather as evidence that the optimized models remain within the accuracy range of their baselines.

The TCN result also demonstrates why a BMS implementation cannot be selected from prediction accuracy or a nominal precision label alone. Although the TCN is the smallest FP32 baseline and achieves the lowest baseline MAE, its current combined optimization path reduces parameter storage by only 29.2%. In particular, quantization provides no further storage reduction after pruning for the evaluated implementation. The achievable benefit therefore depends on the architecture, the supported operators, and the representation generated by the deployment toolchain. Relative compression and absolute deployed footprint must consequently be considered together.

From a system perspective, reduced model storage can create flash headroom for firmware, diagnostics, communication functions, and future model updates. However, parameter count and stored model size do not directly determine runtime RAM, latency, or energy consumption. These quantities must be evaluated on the target hardware because conversion overhead and operator support can offset theoretical savings. The final BMS decision should therefore

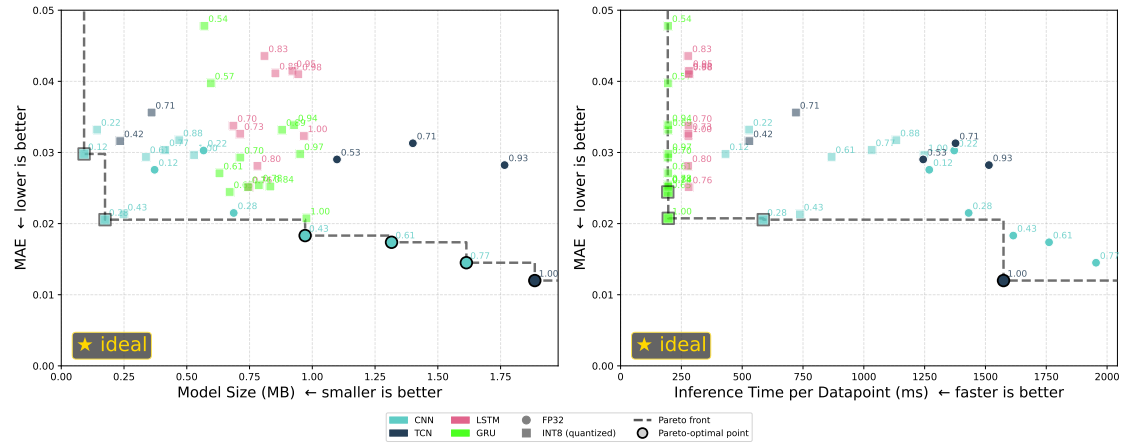


Figure 12: Overall pareto.

combine SOH accuracy with measured flash, RAM, latency, and energy rather than treating pruning or quantization as universally beneficial. In addition, the neural estimate supports supervisory BMS functions and should be integrated with plausibility checks and existing protection mechanisms rather than used as an isolated safety layer.

Figure Directory (Working Draft)

- Figure 1: Study concept from baseline training to compression and embedded evaluation.
- Figure 2: Literature positioning map and research gap framing.
- Figure 3: SOH trajectories with train/validation/test split highlighting.
- Figure 4: Architecture panel for LSTM, GRU, CNN, and TCN.
- Figure 5: Training and baseline-selection workflow.
- Figure 6: Optimization workflow (pruning, quantization, distillation, combined branch).
- Figure 7: Embedded benchmark setup and metric acquisition flow.
- Figure 8: Baseline MAE/RMSE and footprint comparison.
- Figure 9: Pruning sensitivity and pruning Pareto view.
- Figure 10: Quantization sensitivity and precision-resource trade-off.
- Figure 11: Distillation and combined optimization scatter with non-dominated points.
- Figure 12: Cross-architecture decision map.
- Figure 13: Discussion matrix linking deployment constraints to model-method choices.

Data availability (placeholder)

- Dataset DOI, code repository links, and benchmark scripts will be inserted in the final manuscript version.

References

- [1] B. Lin, Z. Li, Towards world's low carbon development: The role of clean energy, *Applied Energy* 307 (2022) 118160. doi:10.1016/j.apenergy.2021.118160.
URL <https://linkinghub.elsevier.com/retrieve/pii/S0306261921014331>
- [2] K. R. Ngoy, V. T. Lukong, K. O. Yoro, J. B. Makambo, N. C. Chukwuati, C. Ibegbulam, O. Eterigho-Ikelegbe, K. Ukoba, T.-C. Jen, Lithium-ion batteries and the future of sustainable energy: A comprehensive review, *Renewable and Sustainable Energy Reviews* 223 (2025) 115971. doi:10.1016/j.rser.2025.115971.
URL <https://linkinghub.elsevier.com/retrieve/pii/S1364032125006446>
- [3] Y. Chen, Y. Kang, Y. Zhao, L. Wang, J. Liu, Y. Li, Z. Liang, X. He, X. Li, N. Tavajohi, B. Li, A review of lithium-ion battery safety concerns: The issues, strategies, and testing standards, *Journal of Energy Chemistry* 59 (2021) 83–99. doi:10.1016/j.jechem.2020.10.017.
URL <https://linkinghub.elsevier.com/retrieve/pii/S2095495620307075>
- [4] F. N. U. Khan, M. G. Rasul, A. Sayem, N. Mandal, Maximizing energy density of lithium-ion batteries for electric vehicles: A critical review, *Energy Reports* 9 (2023) 11–21. doi:10.1016/j.egy.2023.08.069.
URL <https://linkinghub.elsevier.com/retrieve/pii/S2352484723012118>
- [5] J. Zhang, H. Huang, G. Zhang, Z. Dai, Y. Wen, L. Jiang, Cycle life studies of lithium-ion power batteries for electric vehicles: A review, *Journal of Energy Storage* 93 (2024) 112231. doi:10.1016/j.est.2024.112231.
URL <https://linkinghub.elsevier.com/retrieve/pii/S2352152X24018176>
- [6] M. U. Ali, A. Zafar, S. H. Nengroo, S. Hussain, M. Junaid Alvi, H.-J. Kim, Towards a Smarter Battery Management System for Electric Vehicle Applications: A Critical Review of Lithium-Ion Battery State of Charge Estimation, *Energies* 12 (3) (2019) 446. doi:10.3390/en12030446.
URL <https://www.mdpi.com/1996-1073/12/3/446>
- [7] T. Tao, C. Ji, J. Dai, J. Rao, J. Wang, W. Sun, J. Romagnoli, Data-based health indicator extraction for battery SOH estimation via deep learning, *Journal of Energy Storage* 78 (2024) 109982. doi:10.1016/j.est.2023.109982.
URL <https://linkinghub.elsevier.com/retrieve/pii/S2352152X23033819>
- [8] S. Jin, X. Sui, X. Huang, S. Wang, R. Teodorescu, D.-I. Stroe, Overview of Machine Learning Methods for Lithium-Ion Battery Remaining Useful Lifetime Prediction, *Electronics* 10 (24) (2021) 3126. doi:10.3390/electronics10243126.
URL <https://www.mdpi.com/2079-9292/10/24/3126>
- [9] J. Yao, H. Zhao, J. Kowal, Fast-adaptive early-stage remaining useful life prediction of lithium-ion batteries with meta-learning, *Journal of Power Sources* 660 (2025) 238569. doi:10.1016/j.jpowsour.2025.238569.
URL <https://linkinghub.elsevier.com/retrieve/pii/S037877532502405X>
- [10] M.-K. Tran, M. Mathew, S. Janhunen, S. Panchal, K. Raahemifar, R. Fraser, M. Fowler, A comprehensive equivalent circuit model for lithium-ion batteries, incorporating the effects of state of health, state of charge, and temperature on model parameters, *Journal of Energy Storage* 43 (2021) 103252. doi:10.1016/j.est.2021.103252.
URL <https://linkinghub.elsevier.com/retrieve/pii/S2352152X2100949X>
- [11] H. Hu, C. Liang, X. Huang, H. Mo, C. Zou, S. Tao, ONET: Operator network for randomized and robust battery health estimation using operation condition and cycling data matching, *Journal of Power Sources* 672 (2026) 239592. doi:10.1016/j.jpowsour.2026.239592.
URL <https://linkinghub.elsevier.com/retrieve/pii/S0378775326003423>
- [12] P. Iurilli, C. Brivio, R. E. Carrillo, V. Wood, Physics-Based SoH Estimation for Li-Ion Cells, *Batteries* 8 (11) (2022) 204. doi:10.3390/batteries8110204.
URL <https://www.mdpi.com/2313-0105/8/11/204>

- [13] H. Tu, S. Moura, Y. Wang, H. Fang, Integrating physics-based modeling with machine learning for lithium-ion batteries, *Applied Energy* 329 (2023) 120289. doi:10.1016/j.apenergy.2022.120289.
URL <https://linkinghub.elsevier.com/retrieve/pii/S030626192201546X>
- [14] N. Noura, L. Boulon, S. Jemeï, A Review of Battery State of Health Estimation Methods: Hybrid Electric Vehicle Challenges, *World Electric Vehicle Journal* 11 (4) (2020) 66. doi:10.3390/wevj11040066.
URL <https://www.mdpi.com/2032-6653/11/4/66>
- [15] C. Chang, S. Wang, C. Tao, J. Jiang, Y. Jiang, L. Wang, An improvement of equivalent circuit model for state of health estimation of lithium-ion batteries based on mid-frequency and low-frequency electrochemical impedance spectroscopy, *Measurement* 202 (2022) 111795. doi:10.1016/j.measurement.2022.111795.
URL <https://linkinghub.elsevier.com/retrieve/pii/S0263224122009976>
- [16] G. R. Sylvestrin, J. N. Maciel, M. L. M. Amorim, J. P. Carmo, J. A. Afonso, S. F. Lopes, O. H. Ando Junior, State of the Art in Electric Batteries' State-of-Health (SoH) Estimation with Machine Learning: A Review, *Energies* 18 (3) (2025) 746. doi:10.3390/en18030746.
URL <https://www.mdpi.com/1996-1073/18/3/746>
- [17] Z. Yang, Y. Li, Y. Sun, X. Feng, R. Pan, Y. Zhao, State of Health Estimation of Microgrid Energy Storage Batteries Based on An Improved Temporal Convolutional Network, in: *2024 IEEE PES 16th Asia-Pacific Power and Energy Engineering Conference (APPEEC)*, IEEE, Nanjing, China, 2024, pp. 1–5. doi:10.1109/APPEEC61255.2024.10922341.
URL <https://ieeexplore.ieee.org/document/10922341/>
- [18] J. Schulze, N. Strassenburg, T. Rabl, PQ Bench: Benchmarking Pruning and Quantization Techniques, in: *Proceedings of the Workshop on Data Management for End-to-End Machine Learning*, ACM, Berlin Germany, 2025, pp. 1–6. doi:10.1145/3735654.3735940.
URL <https://dl.acm.org/doi/10.1145/3735654.3735940>
- [19] D. P. Pau, A. Aniballi, Tiny Machine Learning Battery State-of-Charge Estimation Hardware Accelerated, *Applied Sciences* 14 (14) (2024) 6240, read_Status: To Read Read_Status_Date: 2026-08-06T10:26:11.747Z. doi:10.3390/app14146240.
URL <https://www.mdpi.com/2076-3417/14/14/6240>
- [20] F. Rzepka, Design of Experiments for Cyclic Aging of LFP 18650 Cells: Impact of Charge/Discharge C-Rates, Depth of Discharge, and Temperature on Battery Degradation (Nov. 2025). doi:<https://doi.org/10.14279/depositonce-24800>.
- [21] C. Nguyen Van, D. T. Quang, Estimation of SoH and internal resistances of Lithium ion battery based on LSTM network, *International Journal of Electrochemical Science* 18 (6) (2023) 100166. doi:10.1016/j.ijoes.2023.100166.
URL <https://linkinghub.elsevier.com/retrieve/pii/S1452398123001931>
- [22] G. Xu, J. Xu, Y. Zhu, LSTM-based estimation of lithium-ion battery SOH using data characteristics and spatio-temporal attention, *PLOS ONE* 19 (12) (2024) e0312856. doi:10.1371/journal.pone.0312856.
URL <https://dx.plos.org/10.1371/journal.pone.0312856>
- [23] L. Ungurean, M. V. Micea, G. Cârstoiu, Online state of health prediction method for lithium-ion batteries, based on gated recurrent unit neural networks, *International Journal of Energy Research* 44 (8) (2020) 6767–6777, read_Status: To Read Read_Status_Date: 2026-08-05T14:41:17.912Z. doi:10.1002/er.5413.
URL <https://onlinelibrary.wiley.com/doi/10.1002/er.5413>
- [24] D. Vaishnani, R. Vaghela, J. Sarda, A. Thakkar, Y. Nasit, A. Khokhariya, A Deep Learning Approach for State of Health Estimation in Lithium-ion Batteries, in: *2024 International Conference on Modeling, Simulation & Intelligent Computing (MoSICom)*, IEEE, Dubai, United Arab Emirates, 2024, pp. 385–390. doi:

- 10.1109/MoSICom63082.2024.10880965.
URL <https://ieeexplore.ieee.org/document/10880965/>
- [25] J. Bi, J.-C. Lee, H. Liu, Performance Comparison of Long Short-Term Memory and a Temporal Convolutional Network for State of Health Estimation of a Lithium-Ion Battery using Its Charging Characteristics, *Energies* 15 (7) (2022) 2448. doi:10.3390/en15072448.
URL <https://www.mdpi.com/1996-1073/15/7/2448>
- [26] A. Aliberti, F. Boni, A. Perol, M. Zampolli, R. J. P. Jaboeuf, P. Tosco, E. Macii, E. Patti, Comparative Analysis of Neural Networks Techniques for Lithium-ion Battery SOH Estimation, in: 2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC), IEEE, Los Alamitos, CA, USA, 2022, pp. 1355–1361. doi:10.1109/COMPSAC54236.2022.00214.
URL <https://ieeexplore.ieee.org/document/9842676/>
- [27] T. Kim, Y. Seo, S. Barde, Edge-compatible SOH estimation for Li-ion batteries via hybrid knowledge distillation and model compression, *Journal of Energy Storage* 135 (2025) 118275. doi:10.1016/j.est.2025.118275.
URL <https://linkinghub.elsevier.com/retrieve/pii/S2352152X25029883>
- [28] Product Specification JGCFR18650-1800mAh-3.2V.
- [29] D. Kucevic, B. Tepe, S. Englberger, A. Parlikar, M. Mühlbauer, O. Bohlen, A. Jossen, H. Hesse, Standard battery energy storage system profiles: Analysis of various applications for stationary energy storage systems using a holistic simulation framework, *Journal of Energy Storage* 28 (2020) 101077. doi:10.1016/j.est.2019.101077.
URL <https://linkinghub.elsevier.com/retrieve/pii/S2352152X19309016>